

Student

So far I have been querying a single table at a time. But real data is split across multiple tables. How do I combine them?

ChatGPT • AI Assistant

CONCEPT CARD

That is exactly what **JOINS** are for. A JOIN combines rows from two or more tables based on a matching condition. The join type decides what happens when there is no match.

Why is data split across tables?

Because of **normalization** — storing data without repetition. Instead of writing the customer's city in every order row, you store it once in the customers table and reference it by `customer_id`. A JOIN reconnects that data at query time.

🔗 ANALOGY

Imagine two spreadsheets — one with customer details, one with orders. A JOIN is like doing a VLOOKUP in Excel: match on a common column and bring columns from both sheets into one combined result.

🌐 REAL WORLD APPLICATION

Every production SQL query in analytics, reporting, and backend development joins multiple tables. Mastering JOINS is the single skill that separates beginners from competent SQL developers.

Student

What schema are we using in this section?

ChatGPT • AI Assistant

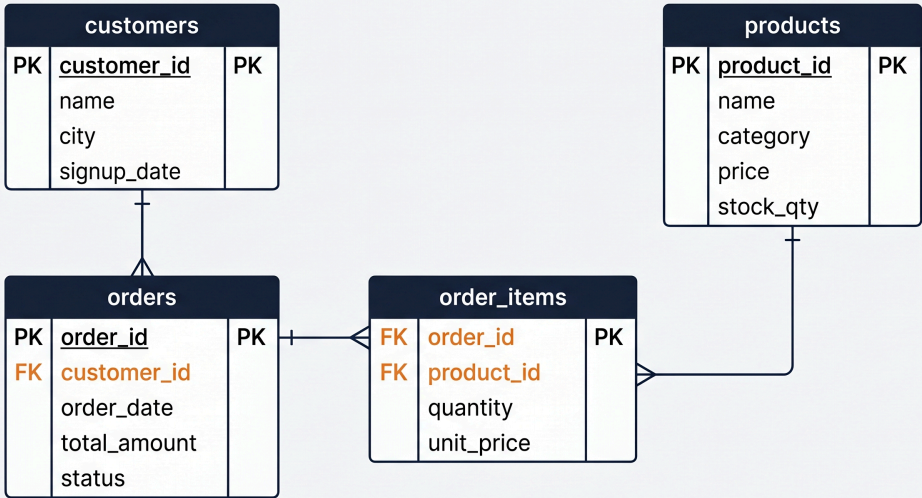
CONCEPT CARD

We introduce a fourth table this section:

- `customers(customer_id, name, city, signup_date)`

- `orders(order_id, customer_id, order_date, total_amount, status)`
- `products(product_id, name, category, price, stock_qty)`
- `order_items(order_id, product_id, quantity, unit_price)`

`order_items` is the bridge table connecting orders to products — each row represents one product line in one order.



`order_items` is the bridge table — it connects orders to products via a many-to-many relationship.

Four-Table E-Commerce Schema with Join Relationships

SQL JOIN Types — Quick Reference				
JOIN Type	Returns	Unmatched Left Rows	Unmatched Right Rows	Use Case
INNER JOIN	Only matching rows from both tables	Excluded	Excluded	Customers who have placed orders
LEFT JOIN	All left rows + matching right rows	Included (NULLs for right cols)	Excluded	All customers including those with no orders
RIGHT JOIN	All right rows + matching left rows	Excluded	Included (NULLs for left cols)	Rare — flip tables and use LEFT JOIN instead
FULL OUTER JOIN	All rows from both tables	Included (NULLs for right cols)	Included (NULLs for left cols)	Find unmatched rows on both sides

JOIN Type	Returns	Unmatched Left Rows	Unmatched Right Rows	Use Case
CROSS JOIN	Cartesian product — every combination	N/A	N/A	Generate combinations (rarely needed)
SELF JOIN	Table joined to itself	Depends on join type used	Depends on join type used	Hierarchical data — employee and manager

sql – JOIN Syntax Template – All Types

```
-- INNER JOIN
SELECT c.name, o.order_id, o.total_amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- LEFT JOIN
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;

-- FULL OUTER JOIN
SELECT c.name, o.order_id
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;

-- SELF JOIN
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

INNER JOIN — RETURN ONLY MATCHING ROWS FROM BOTH TABLES

A two-part diagram. Top half: a classic Venn diagram with two overlapping circles. Left circle labeled 'customers (500 rows)' in light blue. Right circle labeled 'orders (4821 rows)' in light orange. The overlapping center section is filled in dark green with label 'INNER JOIN result — only rows matching on customer_id.' Left-only section (blue, no orange) labeled 'Customers with no orders — EXCLUDED.' Right-only section (orange, no blue) labeled 'Orders with no customer match — EXCLUDED.' Bottom half: a row-level illustration showing 4 customer rows on the left and 6 order rows on the right, with colored match arrows connecting customer_id 101 to two order rows (one-to-many), customer_id 102 to one order row, and customer_id 104 having no matching order row grayed out with a strikethrough and label 'not included.' Clean flat design, sans-serif font.

INNER JOIN — Venn Diagram and Row Matching

sql – INNER JOIN – From Basic to Four-Table Chain

```
-- Basic two-table INNER JOIN
SELECT c.name          AS customer_name,
       c.city,
       o.order_id,
       o.total_amount,
       o.status
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- Three-table INNER JOIN
SELECT c.name          AS customer_name,
       o.order_id,
       p.name          AS product_name,
       oi.quantity
FROM customers c
INNER JOIN orders o      ON c.customer_id = o.customer_id
INNER JOIN order_items oi ON o.order_id   = oi.order_id
INNER JOIN products p    ON oi.product_id = p.product_id;

-- INNER JOIN + WHERE + GROUP BY (combined)
SELECT c.city,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(o.total_amount)        AS city_revenue
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'delivered'
GROUP BY c.city
ORDER BY city_revenue DESC;

-- Join on multiple conditions
SELECT o.order_id, oi.product_id, oi.quantity
FROM orders o
INNER JOIN order_items oi
```

```
ON o.order_id = oi.order_id
AND o.status = 'delivered';
```

Student

Let's start with the most common join. What does INNER JOIN do?

ChatGPT • AI Assistant

CONCEPT CARD

INNER JOIN returns only the rows where the join condition matches in **both** tables. If a row in the left table has no matching row in the right table — or vice versa — that row is excluded from the result entirely.

sql – Code Example

```
SELECT columns
FROM table_a
INNER JOIN table_b ON table_a.key = table_b.key;
```

🔗 ANALOGY

Imagine two lists — a guest list and a check-in list. INNER JOIN gives you only the guests who appear on BOTH lists — people who were invited AND actually checked in. Anyone on only one list is left out.

EXAMPLE CARD**Example 1 — Basic INNER JOIN: Customers and their orders**

sql – Code Example

```
SELECT c.name          AS customer_name,
       c.city,
       o.order_id,
       o.total_amount,
       o.status
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

Output (sample):

code – Code Example

customer_name	city	order_id	total_amount	status
Amelia Watson	Mumbai	1001	1850.00	delivered
Amelia Watson	Mumbai	1047	920.00	shipped
Rohan Mehta	Delhi	1003	2200.00	pending

Notice: Amelia has two orders — she appears twice. Customers with no orders do NOT appear at all.

Example 2 — Three-table INNER JOIN: Orders with customer city and product names

sql – Code Example

```
SELECT c.name          AS customer_name,
       c.city,
       o.order_id,
       o.order_date,
       p.name          AS product_name,
       oi.quantity,
       oi.unit_price
FROM customers c
INNER JOIN orders o      ON c.customer_id = o.customer_id
INNER JOIN order_items oi ON o.order_id   = oi.order_id
INNER JOIN products p   ON oi.product_id = p.product_id;
```

This is a four-table join — the full e-commerce schema stitched together. Chain joins by adding one INNER JOIN at a time, each with its own ON condition.

Example 3 — INNER JOIN with WHERE and aggregate

sql – Code Example

```
SELECT c.city,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(o.total_amount)        AS city_revenue
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'delivered'
GROUP BY c.city
ORDER BY city_revenue DESC;
```

Everything from SQL 3.1 and 3.2 works inside a JOIN query — WHERE, GROUP BY, HAVING, ORDER BY, LIMIT all apply normally.

Example 4 — Joining on multiple conditions

sql — Code Example

```
SELECT *
FROM orders o
INNER JOIN order_items oi
  ON o.order_id = oi.order_id
 AND o.status   = 'delivered'; -- additional condition on the join
```

You can add extra conditions to the ON clause using AND — this restricts which rows are matched during the join.

Student

Do I need to write INNER? Can I just write JOIN?

ChatGPT • AI Assistant

Yes — **JOIN** without a keyword defaults to **INNER JOIN**. Both are identical:

sql — Code Example

```
FROM customers c JOIN orders o ON c.customer_id = o.customer_id;
-- is exactly the same as:
FROM customers c INNER JOIN orders o ON c.customer_id = o.customer_id;
```

Most developers write just **JOIN** for inner joins and spell out **LEFT JOIN**, **RIGHT JOIN**, **FULL OUTER JOIN** explicitly to make the intent clear.

⚠ WARNING

Always use table alias prefixes on every column when joining — e.g. **c.name**, **o.order_id**. Without prefixes, if both tables have a column with the same name (like **customer_id** exists in both customers and orders), the database throws an ambiguous column error.

❗ COMMON PITFALL / MISTAKE

✗ **Ambiguous column — no table prefix:**

sql — Code Example

```
SELECT name, order_id, customer_id    -- which table's customer_id?
FROM customers
JOIN orders ON customers.customer_id = orders.customer_id;
-- ERROR: column 'customer_id' is ambiguous
```

✓ **Correct — always prefix:**

sql — Code Example

```
SELECT c.name, o.order_id, o.customer_id
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

 INTERVIEW CARD

Frequency: Very High

Question: What does INNER JOIN return? What happens to rows that have no match?

Expected Answer: INNER JOIN returns only the rows where the join condition evaluates to TRUE in both tables. Rows in the left table with no matching row in the right table are excluded. Rows in the right table with no matching row in the left table are also excluded. If you need unmatched rows to appear, you must use LEFT JOIN, RIGHT JOIN, or FULL OUTER JOIN.

Follow-up: What is the difference between writing JOIN and INNER JOIN? (Answer: Identical — JOIN defaults to INNER JOIN. Explicit INNER JOIN is just more readable.)

 PRACTICE QUIZ

1. What does INNER JOIN return when a customer has no orders?
2. How do you chain three tables in a single query using INNER JOIN?
3. Predict: if customers has 500 rows and orders has 4821 rows, what is the maximum possible row count from an INNER JOIN between them on customer_id?
4. Fix this: `SELECT name, order_id FROM customers JOIN orders ON customer_id = customer_id;`

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. The customer does not appear in the result — INNER JOIN excludes rows with no match.
2. Add a second INNER JOIN clause after the first: `FROM a INNER JOIN b ON a.id = b.id INNER JOIN c ON b.id = c.id`
3. 4821 — one row per order (assuming every order has a valid customer_id). Could be less if some orders reference non-existent customers.
4. `SELECT c.name, o.order_id FROM customers c JOIN orders o ON c.customer_id = o.customer_id;`

❓ CHECKPOINT QUESTION

Write a query using INNER JOIN that shows the product name, category, quantity ordered, and unit price for all items in order number 1001.

LEFT JOIN — KEEP ALL LEFT ROWS INCLUDING UNMATCHED

ChatGPT • AI Assistant

A side-by-side comparison diagram with two panels. Left panel (labeled 'INNER JOIN'): a Venn diagram with two circles — only the overlapping center section is filled green with label 'Rows returned.' Both non-overlapping sections are grayed out and labeled 'Excluded.' Below the Venn, a small result table shows 3 rows — only the matched customers and orders. Right panel (labeled 'LEFT JOIN'): the same Venn diagram but the entire left circle is highlighted — both the overlapping section (green) and the left-only section (blue) are filled. The right-only section remains grayed out. Below the Venn, a result table shows 5 rows — the 3 matched rows plus 2 additional rows where right-table columns show NULL. A red annotation reads: 'Unmatched left rows → right columns filled with NULL.' Clean flat design, consistent color coding.

LEFT JOIN vs INNER JOIN — Row Inclusion Comparison

sql — LEFT JOIN — Basic, Aggregation, and Multi-Table

```
-- Basic LEFT JOIN: all customers including those with no orders
SELECT c.name          AS customer_name,
       c.city,
       o.order_id,
       o.total_amount
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;

-- LEFT JOIN with aggregation: order count per customer (0 for no-order customers)
SELECT c.name,
       c.city,
```

```
COUNT(o.order_id)                AS order_count,
COALESCE(SUM(o.total_amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_spent DESC;

-- Three-table LEFT JOIN
SELECT c.name,
       o.order_id,
       p.name      AS product_name,
       oi.quantity
FROM customers c
LEFT JOIN orders o      ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id    = oi.order_id
LEFT JOIN products p    ON oi.product_id = p.product_id;

-- RIGHT JOIN rewritten as LEFT JOIN (preferred)
-- Instead of: FROM customers c RIGHT JOIN orders o ...
-- Write:      FROM orders o LEFT JOIN customers c ...
SELECT c.name, o.order_id
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id;
```

Student

INNER JOIN drops customers who have never ordered. What if I want ALL customers — even the ones with no orders?

ChatGPT • AI Assistant

CONCEPT CARD

LEFT JOIN keeps **every row from the left table** regardless of whether there is a match in the right table. For left rows with no match, the right table columns are filled with **NULL**.

sql – Code Example

```
SELECT columns
FROM left_table
LEFT JOIN right_table ON left_table.key = right_table.key;
```

🔗 ANALOGY

Back to the guest list example. LEFT JOIN gives you every guest on the left list. For guests who also checked in (matched), you get their check-in details. For guests who never checked in (no match), their check-in columns are blank — NULL.

EXAMPLE CARD

Example 1 — Basic LEFT JOIN: All customers, including those with no orders

sql – Code Example

```
SELECT c.name          AS customer_name,
       c.city,
       o.order_id,
       o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Output (sample):

code – Code Example

customer_name	city	order_id	total_amount
Amelia Watson	Mumbai	1001	1850.00
Amelia Watson	Mumbai	1047	920.00
Rohan Mehta	Delhi	1003	2200.00
Sara Khan	Hyderabad	NULL	NULL
Priya Nair	Mumbai	NULL	NULL

Sara Khan and Priya Nair have never ordered — they appear with NULL for all orders columns.

Example 2 — LEFT JOIN with aggregation: Total orders per customer including zero-order customers

sql – Code Example

```
SELECT c.name,
       c.city,
       COUNT(o.order_id) AS order_count,
       COALESCE(SUM(o.total_amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
```

```
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_spent DESC;
```

Output (sample):

code – Code Example

name	city	order_count	total_spent
Amelia Watson	Mumbai	8	14200.00
Rohan Mehta	Delhi	5	9800.00
Sara Khan	Hyderabad	0	0.00
Priya Nair	Mumbai	0	0.00

⚠ WARNING

Notice `COUNT(o.order_id)` not `COUNT(*)`. For customers with no orders, `COUNT(*)` would return 1 (counts the NULL row) — wrong. `COUNT(o.order_id)` returns 0 because it skips NULLs. Also `COALESCE(SUM( ... ), 0)` converts NULL sum to 0 for display.

Example 3 — LEFT JOIN across three tables

sql – Code Example

```
SELECT c.name,
       o.order_id,
       p.name AS product_name,
       oi.quantity
FROM customers c
LEFT JOIN orders o      ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id    = oi.order_id
LEFT JOIN products p    ON oi.product_id = p.product_id;
```

All customers appear. Those with no orders show NULL for order, product, and quantity columns.

Student

What about RIGHT JOIN? Should I learn that?

RIGHT JOIN keeps all rows from the **right** table and fills NULLs for unmatched left rows. In practice, almost no engineer uses RIGHT JOIN — you can always swap the table order and use LEFT JOIN instead, which is easier to read:

sql — Code Example

```
-- These two queries return identical results:
SELECT c.name, o.order_id
FROM customers c RIGHT JOIN orders o ON c.customer_id = o.customer_id;

-- Preferred — swap tables and use LEFT JOIN:
SELECT c.name, o.order_id
FROM orders o LEFT JOIN customers c ON o.customer_id = c.customer_id;
```

Learn RIGHT JOIN exists, know what it does, but always rewrite as LEFT JOIN in practice.

INTERVIEW CARD

Frequency: Very High

Question: What is the difference between INNER JOIN and LEFT JOIN?

Expected Answer: INNER JOIN returns only rows where the join condition matches in both tables — unmatched rows from either side are excluded. LEFT JOIN returns all rows from the left table. For left rows with no matching right row, the right table columns are filled with NULL. Use LEFT JOIN when you need to preserve all records from the left table regardless of whether a match exists — for example, listing all customers including those who have never placed an order.

Follow-up: In a LEFT JOIN with aggregation, why should you use `COUNT(righttable.id)` instead of `COUNT(*)`? (Answer: `COUNT(*)` counts NULL rows from unmatched left rows as 1 — wrong. `COUNT(righttable.id)` skips NULLs and correctly returns 0 for unmatched rows.)

PRACTICE QUIZ

1. In a LEFT JOIN, what value appears for right-table columns when there is no match?
2. Why is `COUNT(o.order_id)` better than `COUNT(*)` in a LEFT JOIN aggregation?
3. Rewrite this as a LEFT JOIN: `FROM orders o RIGHT JOIN customers c ON o.customer_id = c.customer_id`

4. Predict: a LEFT JOIN between customers (500 rows) and orders (4821 rows)
— what is the minimum possible row count in the result?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. NULL — all right-table columns are NULL for unmatched left rows.
2. COUNT(*) counts the NULL placeholder row as 1. COUNT(o.order_id) skips NULL and returns 0 correctly.
3. `FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id`
4. 500 — at minimum, each customer appears once (if no customer has multiple orders). In reality more rows, since customers with orders appear once per order.

🔍 CHECKPOINT QUESTION

Write a query that shows every customer's name, city, and their total number of orders and total spend. Include customers who have never ordered — they should show 0 for order count and 0.00 for total spend. Sort by total spend descending.

FULL OUTER JOIN AND THE ANTI-JOIN PATTERN

ChatGPT • AI Assistant

A three-step visual walkthrough. Step 1 (left box, blue background): 'LEFT JOIN — bring all customers, fill NULL for unmatched order rows.' Shows a mini table with 5 customer rows — 3 have order data, 2 have NULL in all order columns. Step 2 (center box, orange background): 'WHERE o.order_id IS NULL — keep only the NULL rows.' The 3 matched rows are struck through and grayed out. Only the 2 NULL rows remain highlighted in orange. Step 3 (right box, green background): 'Result — customers who never ordered.' Shows the final 2-row output table with customer columns only. Below the three steps, a red warning box reads: 'NOT IN is DANGEROUS with NULLs — it silently returns zero rows if the subquery contains any NULL. Always use the anti-join pattern instead.' Arrows connect the three steps left to right.

Anti-Join Pattern — How LEFT JOIN + IS NULL Works

sql — FULL OUTER JOIN, Anti-Join, and NOT IN vs LEFT JOIN IS NULL

```
-- FULL OUTER JOIN: all rows both sides
SELECT c.customer_id, c.name, o.order_id
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;

-- FULL OUTER JOIN: find data integrity issues (orphans both sides)
```

```
SELECT c.customer_id, c.name, o.order_id, o.customer_id AS
order_cust_id
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL    -- orphan orders
      OR o.order_id IS NULL;   -- customers with no orders

-- Anti-join: customers who have NEVER ordered
SELECT c.customer_id, c.name, c.city, c.signup_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Anti-join: products never ordered
SELECT p.product_id, p.name, p.category, p.price
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.order_id IS NULL;

-- DANGEROUS: NOT IN breaks with NULLs
-- SELECT * FROM customers WHERE customer_id NOT IN (SELECT customer_id
FROM orders);
-- Returns ZERO rows if any order has NULL customer_id!

-- SAFE: anti-join pattern
SELECT c.*
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Student

What if I want unmatched rows from BOTH sides — not just the left?

ChatGPT • AI Assistant

CONCEPT CARD

FULL OUTER JOIN returns all rows from both tables. Where there is a match, columns from both tables are populated. Where there is no match on either side, the missing table's columns are filled with NULL.

sql – Code Example

```
SELECT c.name, o.order_id
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
```

Result includes:

- Matched rows (customer has an order)
- Left-only rows (customers with no orders — right columns NULL)
- Right-only rows (orders with no matching customer — left columns NULL)

EXAMPLE CARD

Example 1 — FULL OUTER JOIN: Find any data integrity issues

sql – Code Example

```
SELECT c.customer_id AS cust_id,
       c.name         AS customer_name,
       o.order_id,
       o.customer_id AS order_cust_id
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL    -- orphan orders (no matching customer)
     OR o.order_id IS NULL;   -- customers with no orders
```

This reveals both orphan orders (orders referencing deleted customers) and customers who have never ordered — in one query.

Student

What is an anti-join? I keep hearing about that pattern.

ChatGPT • AI Assistant

CONCEPT CARD

An **anti-join** finds rows in one table that have NO match in another table. The classic pattern uses `LEFT JOIN ... WHERE right_table.id IS NULL`.

Example 2 — Anti-join: Customers who have NEVER placed an order

sql – Code Example

```
SELECT c.customer_id,  
       c.name,  
       c.city,  
       c.signup_date  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;  -- no matching order row = never ordered
```

Output:

code — Code Example

customer_id	name	city	signup_date
205	Sara Khan	Hyderabad	2023-08-14
312	Priya Nair	Mumbai	2023-11-02

How does this work? LEFT JOIN brings in all customers. For customers with no orders, `o.order_id` is NULL. The WHERE clause keeps only those NULL rows — exactly the customers with no orders.

🔗 ANALOGY

Imagine stamping every left row — green if it matched, red if it did not. The anti-join is a WHERE clause that keeps only the red-stamped rows.

Student

Why not just use NOT IN?

ChatGPT • AI Assistant

⚠️ WARNING

Using NOT IN is dangerous when the subquery can return NULLs:

⚠️ COMMON PITFALL / MISTAKE

❌ **Dangerous — NOT IN breaks silently with NULLs:**

sql — Code Example

```
SELECT * FROM customers  
WHERE customer_id NOT IN (
```

```
SELECT customer_id FROM orders
); -- returns ZERO rows if ANY order has NULL customer_id!
```

✓ **Safe — anti-join pattern:**

sql – Code Example

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Why does NOT IN break with NULLs? Because $x \text{ NOT IN } (1, 2, \text{NULL})$ evaluates as $x \neq 1 \text{ AND } x \neq 2 \text{ AND } x \neq \text{NULL}$. Since $x \neq \text{NULL}$ is NULL (not TRUE), the entire expression is NULL — no row passes. The anti-join pattern is immune to this because it tests `o.order_id IS NULL` directly.

Example 3 — Anti-join with aggregation: Customers who never ordered, and how long ago they signed up

sql – Code Example

```
SELECT c.name,
       c.city,
       c.signup_date,
       CURRENT_DATE - c.signup_date AS days_since_signup
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY days_since_signup DESC;
```

Example 4 — Products that have never been ordered

sql – Code Example

```
SELECT p.product_id,
       p.name      AS product_name,
       p.category,
       p.price
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.order_id IS NULL;
```

This is a classic inventory query — find products sitting in stock that no customer has ever purchased.

INTERVIEW CARD

Frequency: High

Question: How do you find rows in table A that have no match in table B?

Expected Answer: Use the anti-join pattern: LEFT JOIN table B on the key column, then add WHERE b.key IS NULL in the WHERE clause. This is safer and more efficient than NOT IN with a subquery because NOT IN breaks silently when the subquery returns any NULL values — the entire NOT IN condition evaluates to NULL and zero rows pass. The anti-join pattern avoids this trap entirely.

Follow-up: When would you use FULL OUTER JOIN? (Answer: When you need unmatched rows from BOTH sides — for example, detecting orphan records on both sides of a relationship in a data quality audit.)

PRACTICE QUIZ

1. What does FULL OUTER JOIN return that LEFT JOIN does not?
2. Write the anti-join pattern to find products never ordered.
3. Why is NOT IN dangerous when the subquery can return NULLs?
4. Predict: `SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id WHERE o.order_id IS NULL;` — what does this return?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. FULL OUTER JOIN also returns unmatched rows from the right table (right-only rows). LEFT JOIN only returns unmatched rows from the left.
2. `SELECT p.* FROM products p LEFT JOIN order_items oi ON p.product_id = oi.product_id WHERE oi.order_id IS NULL;`
3. `x NOT IN (a, b, NULL)` includes `x ≠ NULL` which evaluates to NULL — so no row ever passes the filter. Zero rows returned silently.
4. Only customers who have never placed an order — rows where the LEFT JOIN found no matching order, so o.order_id is NULL.

CHECKPOINT QUESTION

Using the anti-join pattern, write a query that finds all customers from Mumbai who signed up in 2023 but have never placed a delivered or shipped order. Show customer name, city, and signup date.

SELF JOIN — JOINING A TABLE TO ITSELF

ChatGPT • AI Assistant

A two-part diagram. Top half: an org chart tree showing a company hierarchy. The root node (CEO level) shows 'Carol Singh — Engineering' with no manager. Two child nodes connect to Carol: 'Alice Sharma — Engineering' and 'Bob Kumar — Engineering'. A second root node shows 'Eve Nair — Marketing' with children 'Dave Rao — Marketing'. Tree lines show reporting relationships. Bottom half: the same data shown as a SQL self join illustration. Two side-by-side table instances labeled 'employees e (employee)' and 'employees m (manager)'. Arrows connect e.manager_id to m.employee_id for each matching row. Carol Singh and Eve Nair have no arrow from the manager side — their m.name shows NULL with the label 'No manager — LEFT JOIN preserves these rows. INNER JOIN would remove them.' Clean diagram, org chart nodes use rounded rectangles.

SELF JOIN — Employee-Manager Org Chart Example

sql — SELF JOIN — Org Chart, Salary Comparison, and Hierarchy

```
-- Basic self join: each employee with their manager
SELECT e.employee_id,
       e.name          AS employee_name,
       e.department,
       m.name          AS manager_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id
ORDER BY e.department, e.name;

-- Employees who earn MORE than their manager (classic interview
question)
SELECT e.name      AS employee,
       e.salary    AS emp_salary,
       m.name      AS manager,
       m.salary    AS mgr_salary
FROM employees e
INNER JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary
ORDER BY e.salary DESC;

-- Employees in same department as their manager
SELECT e.name      AS employee,
       e.department,
       m.name      AS manager
```

```
FROM employees e
INNER JOIN employees m
  ON e.manager_id = m.employee_id
 AND e.department = m.department;

-- Find top-level employees (no manager)
SELECT e.name, e.department
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id
WHERE m.employee_id IS NULL;
```

Student

You mentioned SELF JOIN. That sounds strange — how can a table join to itself?

ChatGPT • AI Assistant

CONCEPT CARD

A **SELF JOIN** joins a table to itself using two different aliases. It is used when rows in a table reference other rows in the same table — the classic example is a hierarchical structure like an employee table where each employee has a `manager_id` that references another employee's `employee_id`.

sql – Code Example

```
SELECT e.name AS employee,
       m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

Here `e` and `m` are two different aliases for the same `employees` table. `e` represents each employee, `m` represents their manager.

🔗 ANALOGY

Imagine a company org chart. The employees table has every person including managers. To find 'who does Alice report to?', you look up Alice's *managerid*, then find that *employeeid* in the same table. A self join does this in SQL automatically for all employees at once.

EXAMPLE CARD

Schema for this subsection:

sql – Code Example

employees(employee_id, name, department, salary, manager_id)
-- manager_id references employee_id in the same table
-- the CEO has manager_id = NULL (no manager)

Example 1 — Basic SELF JOIN: Each employee with their manager's name

sql – Code Example

SELECT e.employee_id,
 e.name AS employee_name,
 e.department,
 m.name AS manager_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id
ORDER BY e.department, e.name;

Output (sample):

code – Code Example

employee_id	employee_name	department	manager_name
101	Alice Sharma	Engineering	Carol Singh
102	Bob Kumar	Engineering	Carol Singh
103	Carol Singh	Engineering	NULL
104	Dave Rao	Marketing	Eve Nair
105	Eve Nair	Marketing	NULL

Carol Singh and Eve Nair are managers with no manager above them — **manager_name** is NULL.

⚠ WARNING

Use **LEFT JOIN** in a self join when the top-level row (like the CEO) has NULL in the self-referencing column — an INNER JOIN would exclude them entirely.

Example 2 — SELF JOIN with filter: Find employees who earn more than their manager

sql – Code Example

```
SELECT e.name      AS employee,  
       e.salary    AS emp_salary,  
       m.name      AS manager,  
       m.salary    AS mgr_salary  
FROM employees e  
INNER JOIN employees m ON e.manager_id = m.employee_id  
WHERE e.salary > m.salary  
ORDER BY e.salary DESC;
```

This is a classic interview question — it requires a self join.

Example 3 — SELF JOIN: List all employee-manager pairs in the same department

sql – Code Example

```
SELECT e.name      AS employee,  
       e.department,  
       m.name      AS manager  
FROM employees e  
INNER JOIN employees m  
  ON e.manager_id = m.employee_id  
 AND e.department = m.department;
```

Student

Is the SELF JOIN just limited to org charts?

ChatGPT • AI Assistant

🌐 REAL WORLD APPLICATION

No — any table where rows reference other rows in the same table needs a self join.
Common examples in production:

- **Categories table** with a `parent_category_id` column (e-commerce product hierarchies)
- **Accounts table** in banking where accounts have sub-accounts
- **Referral systems** where users have a `referred_by_user_id`
- **Thread replies** in forums where a reply has a `parent_post_id`

👤 INTERVIEW CARD

Frequency: Medium-High

Question: Write a SQL query to find all employees who earn more than their direct manager.

Expected Answer:

sql – Code Example

```
SELECT e.name AS employee, e.salary AS emp_salary,  
       m.name AS manager, m.salary AS mgr_salary  
FROM employees e  
INNER JOIN employees m ON e.manager_id = m.employee_id  
WHERE e.salary > m.salary;
```

Use INNER JOIN here because employees without a manager (CEO) should not appear — they have no manager salary to compare against.

Follow-up: Why do you use two different aliases for the same table? (Answer: SQL needs two distinct references to the same table. Without aliases, there is no way to distinguish which instance of the table is the employee and which is the manager.)

🔍 PRACTICE QUIZ

1. Why must you use two aliases in a self join?
2. Should you use INNER JOIN or LEFT JOIN in a self join when the top-level row has NULL manager_id?
3. Predict the output: `SELECT e.name, m.name AS manager FROM employees e LEFT JOIN employees m ON e.manager_id = m.employee_id WHERE m.employee_id IS NULL;` — what does this return?
4. Name two real-world tables other than employees that would need a self join.

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Because the same table is referenced twice — you need two different aliases to distinguish which instance represents the employee and which represents the manager.
2. LEFT JOIN — INNER JOIN would exclude top-level rows with NULL manager_id (like the CEO), removing them from the result.

3. Employees with no manager — i.e., top-level employees (CEO or department heads). The WHERE keeps only rows where the self join found no matching manager row.
4. Any two of: categories table (parentcategoryid), referral/users table (referredbyuserid), forum posts (parentpostid), org chart (managerid), accounts (parentaccountid).

🔍 CHECKPOINT QUESTION

Using the employees table, write a self join query that shows each employee's name, their department, their salary, their manager's name, and their manager's salary — but only for employees in the 'Engineering' department whose salary is within 20% of their manager's salary. Sort by employee salary descending.

CROSS JOIN, ACCIDENTAL CARTESIAN PRODUCTS, AND 10 PRACTICE PROBLEMS

ChatGPT • AI Assistant

A 2x3 grid of Venn diagrams, each showing two overlapping circles labeled 'Table A (left)' and 'Table B (right)'. Each cell illustrates a different join type with the returned section filled in dark green and excluded sections grayed out. Cell 1 (top-left): INNER JOIN — only the overlapping center section is green. Caption: 'Only matched rows.' Cell 2 (top-center): LEFT JOIN — entire left circle filled green. Caption: 'All left rows + matched right.' Cell 3 (top-right): RIGHT JOIN — entire right circle filled green. Caption: 'All right rows + matched left. Rarely used.' Cell 4 (bottom-left): FULL OUTER JOIN — both full circles filled green. Caption: 'All rows both sides.' Cell 5 (bottom-center): LEFT ANTI-JOIN — only the left non-overlapping section is green. Caption: 'Left rows with NO match in right. WHERE b.id IS NULL.' Cell 6 (bottom-right): CROSS JOIN — both circles shown side by side without overlap, both fully green, with a multiplication symbol between them. Caption: 'Every combination. No ON clause. Use carefully.' Consistent color scheme, sans-serif font, clean grid layout.

All JOIN Types — Venn Diagram Summary

sql – 10 Practice Problems – Complete Solutions

```
-- Problem 1: Customer names with order IDs and amounts
SELECT c.name AS customer_name, o.order_id, o.total_amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.name;

-- Problem 2: Product name, quantity, unit price for all order items
SELECT p.name AS product_name, oi.quantity, oi.unit_price
FROM order_items oi
INNER JOIN products p ON oi.product_id = p.product_id
ORDER BY p.name;
```

```
-- Problem 3: All customers and their order count (0 if none)
SELECT c.name, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY order_count DESC;

-- Problem 4: Four-table join, delivered orders only
SELECT c.name AS customer_name, c.city,
       p.name AS product_name,
       oi.quantity,
       oi.quantity * oi.unit_price AS line_total
FROM customers c
INNER JOIN orders o      ON c.customer_id = o.customer_id
INNER JOIN order_items oi ON o.order_id   = oi.order_id
INNER JOIN products p    ON oi.product_id = p.product_id
WHERE o.status = 'delivered'
ORDER BY line_total DESC;

-- Problem 5: Anti-join – customers who never ordered
SELECT c.customer_id, c.name, c.city, c.signup_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY c.signup_date DESC;

-- Problem 6: Anti-join – products never ordered
SELECT p.product_id, p.name AS product_name, p.category, p.price
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.order_id IS NULL
ORDER BY p.category, p.price;

-- Problem 7: Self join – employee with manager name
SELECT e.employee_id, e.name AS employee_name,
       e.department, e.salary,
       m.name AS manager_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id
ORDER BY e.department, e.name;

-- Problem 8: Self join – employees earning more than manager
SELECT e.name AS employee, e.salary AS emp_salary,
       m.name AS manager,  m.salary AS mgr_salary,
       e.salary - m.salary AS salary_diff
FROM employees e
```

```
INNER JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary
ORDER BY salary_diff DESC;

-- Problem 9: Cities with more than 10 never-ordered customers
SELECT c.city, COUNT(*) AS never_ordered_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
GROUP BY c.city
HAVING COUNT(*) > 10
ORDER BY never_ordered_count DESC;

-- Problem 10: Category revenue > 50000, delivered orders only
SELECT p.category,
       SUM(oi.quantity) AS total_units_sold,
       SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM orders o
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p ON oi.product_id = p.product_id
WHERE o.status = 'delivered'
GROUP BY p.category
HAVING SUM(oi.quantity * oi.unit_price) > 50000
ORDER BY total_revenue DESC;
```

Student

What is a CROSS JOIN and when would I use it?

ChatGPT • AI Assistant

CONCEPT CARD

CROSS JOIN returns the **Cartesian product** — every row from the left table combined with every row from the right table. There is no ON condition.

sql – Code Example

```
SELECT a.name, b.category
FROM products a
CROSS JOIN products b;
-- 100 products x 100 products = 10,000 rows
```

🌐 REAL WORLD APPLICATION

CROSS JOIN is rare but has legitimate uses:

- Generating a complete combination grid (e.g., all size × colour combinations for a clothing catalogue)
- Creating a date spine — cross joining a date dimension table with a product table to ensure every product appears for every date even with zero sales

⚠️ WARNING

CROSS JOIN on large tables is catastrophic. 10,000 rows × 10,000 rows = 100 million rows. Always be intentional when writing CROSS JOIN.

⚠️ COMMON PITFALL / MISTAKE

✗ Accidental CROSS JOIN — missing ON condition:

sql — Code Example

```
SELECT c.name, o.order_id
FROM customers c
JOIN orders o;    -- MISSING ON clause!
-- Result: 500 customers x 4821 orders = 2,410,500 rows!
```

✅ Always specify the ON condition:

sql — Code Example

```
SELECT c.name, o.order_id
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

Student

Can we go through all 10 practice problems now?

ChatGPT • AI Assistant

CONCEPT CARD

Here are 10 problems covering all join types. Try each before reading the solution.

Problem 1 — Easy: INNER JOIN — List customer names with their order IDs and total amounts.

sql – Code Example

```
SELECT c.name AS customer_name,  
       o.order_id,  
       o.total_amount  
FROM customers c  
INNER JOIN orders o ON c.customer_id = o.customer_id  
ORDER BY c.name;
```

Problem 2 — Easy: INNER JOIN — Show product name, quantity, and unit price for all items in all orders.

sql – Code Example

```
SELECT p.name AS product_name,  
       oi.quantity,  
       oi.unit_price  
FROM order_items oi  
INNER JOIN products p ON oi.product_id = p.product_id  
ORDER BY p.name;
```

Problem 3 — Easy: LEFT JOIN — Show all customers and their order count (0 for customers with no orders).

sql – Code Example

```
SELECT c.name,  
       COUNT(o.order_id) AS order_count  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.name  
ORDER BY order_count DESC;
```

Problem 4 — Medium: Four-table INNER JOIN — Show customer name, city, product name, quantity, and line total (quantity × unit_price) for all delivered orders.

sql – Code Example

```
SELECT c.name AS customer_name,  
       c.city,  
       p.name AS product_name,  
       oi.quantity,  
       oi.quantity * oi.unit_price AS line_total
```

```
FROM customers c
INNER JOIN orders o      ON c.customer_id = o.customer_id
INNER JOIN order_items oi ON o.order_id   = oi.order_id
INNER JOIN products p    ON oi.product_id = p.product_id
WHERE o.status = 'delivered'
ORDER BY line_total DESC;
```

Problem 5 — Medium: Anti-join — Find all customers who have NEVER placed an order.

sql — Code Example

```
SELECT c.customer_id,
       c.name,
       c.city,
       c.signup_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY c.signup_date DESC;
```

Problem 6 — Medium: Anti-join — Find products that have never been ordered.

sql — Code Example

```
SELECT p.product_id,
       p.name      AS product_name,
       p.category,
       p.price
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.order_id IS NULL
ORDER BY p.category, p.price;
```

Problem 7 — Medium: Self join — Show each employee alongside their manager's name.

sql — Code Example

```
SELECT e.employee_id,
       e.name      AS employee_name,
       e.department,
       e.salary,
       m.name      AS manager_name
```

```
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id
ORDER BY e.department, e.name;
```

Problem 8 — Hard: Self join — Employees who earn more than their manager.

sql – Code Example

```
SELECT e.name      AS employee,
       e.salary    AS emp_salary,
       m.name      AS manager,
       m.salary    AS mgr_salary,
       e.salary - m.salary AS salary_diff
FROM employees e
INNER JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary
ORDER BY salary_diff DESC;
```

Problem 9 — Hard: LEFT JOIN + GROUP BY + HAVING — Cities where the total number of customers who have NEVER ordered is greater than 10.

sql – Code Example

```
SELECT c.city,
       COUNT(*) AS never_ordered_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
GROUP BY c.city
HAVING COUNT(*) > 10
ORDER BY never_ordered_count DESC;
```

Problem 10 — Hard: Four-table join with GROUP BY and HAVING — Product categories with total revenue above ₹50,000 from delivered orders only, showing category, total units sold, and total revenue.

sql – Code Example

```
SELECT p.category,
       SUM(oi.quantity)           AS total_units_sold,
       SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM orders o
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p    ON oi.product_id = p.product_id
```

```
WHERE o.status = 'delivered'  
GROUP BY p.category  
HAVING SUM(oi.quantity * oi.unit_price) > 50000  
ORDER BY total_revenue DESC;
```

SUMMARY CARD

SQL 1.3 Key Takeaways:

- **INNER JOIN** — returns only matched rows. Unmatched rows from either side are excluded.
- **LEFT JOIN** — returns all left rows. Unmatched left rows get NULL for right-side columns.
- **RIGHT JOIN** — returns all right rows. Almost never used — rewrite as LEFT JOIN by swapping tables.
- **FULL OUTER JOIN** — returns all rows from both sides. Unmatched rows on either side get NULLs.
- **Anti-join pattern** — `LEFT JOIN ... WHERE right_table.id IS NULL` — finds rows with no match.
- **NOT IN is dangerous** with NULLs — use the anti-join pattern instead.
- **SELF JOIN** — a table joined to itself using two aliases — used for hierarchical data.
- **CROSS JOIN** — Cartesian product — use intentionally and carefully.
- **Always use table alias prefixes** on every column in a join query — prevents ambiguous column errors.
- **Always specify the ON condition** — a missing ON clause produces an accidental CROSS JOIN.

🔮 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior SQL mentor. I just learned INNER JOIN, LEFT JOIN, FULL OUTER JOIN, anti-join, and SELF JOIN. Give me 10 progressively harder JOIN problems using an e-commerce schema with tables: customers(customerid, name, city, signupdate), orders(orderid, customerid, orderdate, totalamount, status), products(productid, name, category, price, stockqty), orderitems(orderid, productid, quantity, unitprice). After each problem: wait for my answer, show the correct SQL, explain every join decision, highlight the most common mistake students make on that problem, and give one interview question related to it. Do not give me the next problem until I answer.'

🔍 CHECKPOINT QUESTION

Before moving to SQL 1.4 (Subqueries): without running it, identify every mistake in this query and write the fully corrected version:

sql – Code Example

```
SELECT name, order_id, total_amount, product_name
FROM customers
JOIN orders ON customer_id = customer_id
JOIN order_items ON order_id = order_id
JOIN products ON product_id = product_id
WHERE status = 'delivered'
AND total_amount > 1000;
```

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.3 — joins, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Four-Table E-Commerce Schema with Join Relationships:** An entity-relationship diagram showing four rectangular table boxes with all col...